Retrieval Augmented Generation Fundamentals

This guide explains how a Retrieval Augmented Generation system, usually called RAG, gives a large language model access to private knowledge. It covers ingestion, chunking and the retrieval structures that make answers accurate.

1. Why RAG Exists

Large language models learn from a fixed snapshot of public text. They do not know your company handbook, your product documentation or yesterday's support tickets. When you ask about that material, the model either refuses or invents an answer. Inventing an answer is called a hallucination, and it is the main reason teams distrust AI assistants.

RAG fixes this by looking up relevant text first and placing it in the prompt. The model then answers from the supplied text instead of from memory. Because the knowledge lives outside the model, you can update it in seconds without any training.

A RAG system has two phases. The ingestion phase runs once per document. The query phase runs every time a user asks a question. The quality of the query phase depends almost entirely on how well the ingestion phase was done.

2. The Ingestion Pipeline

Ingestion turns raw files into small, clean, searchable pieces of knowledge. A reliable pipeline follows the same steps for every document:

- Parse the file and extract its text.
- Convert the text to Markdown so headings and lists survive.
- Extract metadata such as the title, author and page count.
- Clean the text by removing page numbers, headers and footers.
- Split the cleaned text into chunks.

2.1 Parsing and Markdown

PDF files store characters at fixed positions on a page. They have no concept of a paragraph or a heading. A parser must rebuild that structure from font sizes and spacing. Markdown is a good target format because headings, lists and tables stay readable for both people and language models.

2.2 Cleaning

Raw PDF text is noisy. Every page repeats the same header and footer, and page numbers appear between paragraphs. If these lines reach the vector store, they pollute search results. A cleaning step removes them before chunking so every chunk contains only real content.

3. Chunking Strategies

A language model can only read a limited amount of text at once, and search works best on small focused passages. Chunking is the process of cutting a document into pieces that are small enough to search and large enough to make sense on their own.

3.1 Recursive Chunking

Recursive chunking tries the largest natural boundary first. It splits on headings, then paragraphs, then sentences, and finally words. A piece is only split further when it is still larger than the chunk size. Neighbouring small pieces are merged so chunks stay close to the target length. A small overlap between chunks protects sentences that would otherwise be cut in half.

Recursive chunking is fast, predictable and works well on almost any document. It is the default choice in most production systems.

3.2 Semantic Chunking

Semantic chunking groups text by meaning instead of by length. Paragraphs that discuss the same idea stay together, and a new chunk starts when the topic changes. Advanced versions compare embeddings of neighbouring sentences. A simpler version uses rules, such as keeping a list with the sentence that introduces it and starting a new chunk at every heading.

The benefit is that each chunk answers one question well. The cost is that chunk sizes vary, so some chunks are much larger than others.

4. Parent Child Retrieval

Small chunks are good for search because their meaning is focused. Large chunks are good for answering because they contain full context. Parent child retrieval gives you both. The document is split into large parents, usually one per section. Each parent is then split into small children.

At query time the system searches the children. When a child matches, the system returns its parent to the language model. The model sees the full section, but the search was precise. This structure is the foundation for the retrieval videos that follow this lesson.

5. Choosing Chunk Sizes

There is no perfect chunk size. Use these guidelines as a starting point and measure the results on your own questions:

- Children between 300 and 500 characters work well for precise search.
- Parents between 1500 and 3000 characters give the model enough context.
- Overlap of 10 to 15 percent protects sentences at chunk boundaries.
- Always keep headings attached to the text that follows them.

Finally, remember that chunking is not a one time decision. Log which chunks were retrieved for real questions, review the failures and adjust the settings. Small changes to chunk size often improve answer quality more than switching to a bigger model.